

AI-Driven Energy Supply Chain Resilience for India

A Digital Twin, Adaptive Procurement Orchestrator, and Disruption Scenario Modeller for India's Crude Oil Import Network

Technical Project Report · Prepared by DeadlineDriven · July 2026

Executive Summary

This project is a working prototype developed for the ET AI Hackathon 2026 Problem Statement 2. This statement calls for an AI-driven system that alerts India's energy planners about possible crude-oil supply disruptions and models a flexible response. The submission includes a 71-node, 65-edge supply-chain knowledge graph based entirely on cited PPAC and DGCIS government data; a disruption scenario engine that examines four chokepoint scenarios; a two-stage stochastic linear program with a CVaR risk-aversion term, which recommends a procurement response in under one second; and a five-tab interactive Streamlit dashboard that presents all this information in clear language while also providing full technical detail.

The build is validated and not just internally consistent. Its key figures, such as the Hormuz-routed import share, capacity lost during closures, and strategic reserve cover are compared directly to actual reports on the Strait of Hormuz closure that began on 28 February 2026 and fall within or very close to independently reported ranges. The codebase includes 111 automated tests, and an independent audit of the solved model identified three real modeling bugs. All these issues were fixed and turned into permanent regression tests, which this report views as a sign of thoroughness, not a shortcoming to downplay.

What follows documents the problem, the architecture, the data methodology, the exact mathematical formulation of the optimizer, the validation process, the engineering steps, and clearly outlines what has not been built yet and why. This way, scope decisions are transparent rather than hidden.

1. Problem Statement & Motivation

India imports around 89.7% of the crude oil it refines, 4,936 kbpd in FY2025-26, compared to 562 kbpd from domestic production. A significant share of those imports, 48.2%, passes through the Strait of Hormuz. PS2 describes the main challenge as building an AI system that (a) detects geopolitical and market signals indicating a disruption, (b) models the effects of that disruption on the supply network, and (c) suggests ways to change procurement and reserve use in response, in a matter of days rather than months.

This is not just a theoretical exercise for this development. As noted in Section 7, the Strait of Hormuz has been effectively closed to commercial shipping since February 28, 2026, and remains closed as of now. This was confirmed through live search rather than from previous training data since the event occurred after any knowledge cutoff. The worst-case scenario this system examines is the current, ongoing situation, which is why the validation exercise in Section 7 is significant and not merely a synthetic stress test.

2. Solution Overview: Coverage of PS2's Illustrative Components

PS2 lists five illustrative components a team might build. The table below states plainly which are built, which are scoped down, and why every scope decision here is also documented in KNOWN_LIMITATIONS.md in the repository.

PS2 Component	Status	What Exists Today
Supply Chain Digital Twin	Built	71-node / 65-edge knowledge graph (data/network.json) + 5-tab interactive Streamlit dashboard, fully PPAC/DGCI&S sourced.
Disruption Scenario Modeller	Built	Four chokepoint scenarios (baseline, Hormuz 50%, Hormuz 100%, Red Sea/Suez 80%) degrade live edge capacities by severity × chokepoint exposure.
Adaptive Procurement Orchestrator	Built	Two-stage stochastic linear program with a CVaR risk-aversion term, solved in under a second with Pyomo + HiGHS. Full formulation in Section 5.
Geopolitical Risk Intelligence Agent	Scoped down	signal_extractor.py maps curated, real, dated 2026 headlines to a severity/corridor signal. Not yet live multi-source (RSS/API) monitoring.
Strategic Reserve Optimisation Agent	Scoped down	SPR drawdown is modeled per scenario as a single representative day (capped by max daily draw rate), not a multi-day depletion-and-replenishment schedule.

3. System Architecture

The system has five layers. Source documents are converted once, offline, into a typed knowledge graph. Three engines read that graph to extract signals, degrade it under a scenario, and solve a procurement allocation problem based on it. A plain-language translation layer sits between the optimizer and the UI. A five-tab Streamlit app serves as the single presentation surface for everything.

ARCHITECTURE

From source documents to a decision, in five layers

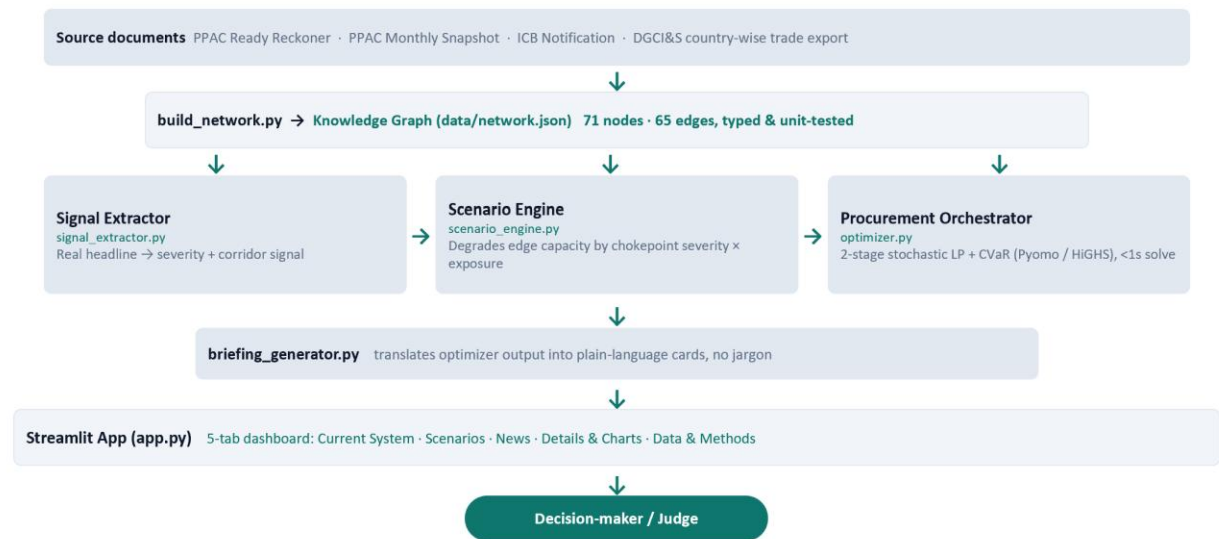


Figure 1. End-to-end architecture, source documents to decision-maker.

Every box in Figure 1 corresponds to a real, versioned file in the repository `build_network.py`, `data/network.json`, `signal_extractor.py`, `scenario_engine.py`, `optimizer.py`, `briefing_generator.py`, and `app.py` there is no component here that exists only as a diagram.

4. Data Foundation & Methodology

The knowledge graph is built once, offline, by `build_network.py`, from four user-provided source documents: PPAC's India Oil & Gas Ready Reckoner FY2025-26, PPAC's Monthly Snapshot (June 2026 data), the ICB Notification dated 30.06.2026, and a DGCI&S Foreign Trade Data Dissemination Portal export covering calendar-2025 country-wise crude imports (34 reporting countries). None of these source files are parsed at runtime the figures they contain are extracted once and hard-coded as cited constants, which keeps the running app fast and its provenance auditable.

Metric	Value	Source
Total nodes / edges	71 / 65	<code>build_network.py</code> output, <code>data/network.json</code>
Refineries modeled	16 (~97% of national capacity)	PPAC Ready Reckoner Table 4.1
Supplier nodes	37 (33 countries + 4 domestic regions)	DGCI&S country-wise import register
Ports / corridors / SPR sites	11 / 4 / 3	<code>network.json</code> typed nodes
Crude imports, FY2025-26	4,936.2 kbpd	PPAC Snapshot, June 2026 issue
Indigenous production, FY2025-26	562.3 kbpd	PPAC Snapshot, June 2026 issue
India Basket price, FY2025-26 avg.	\$70.99/bbl	PPAC Snapshot, Table 25
Strategic reserve (combined)	5.33 MMT, ~9.5 days at full fill	PIB/Wikipedia (not PPAC see Section 8)

A country-wise supplier split was originally pooled into a single 'Other' node for the 26 countries each individually under 2% of import share. Following an internal review, all 25 countries with nonzero recorded volume were separated into individually modeled nodes, each with its own real export terminal, corridor assignment, and shipping edge raising the node count from 47 to 71 and closing a routing-approximation gap the pooled node had introduced (documented in `KNOWN_LIMITATIONS.md` §9).

5. Model Formulation: Adaptive Procurement Orchestrator

The optimizer (`optimizer.py`) is a two-stage stochastic linear program every decision variable is continuous; there are no integer or binary variables, which keeps the full-network solve under a second even behind an interactive slider. It is solved with Pyomo against the open-source HiGHS solver.

5.1 Decision Variables

Variable	Stage	Meaning
<code>contract[e]</code>	First-stage	Volume contracted on shipping edge <code>e</code> , fixed before the scenario is known.
<code>shipping_flow[e,ω]</code>	Second-stage	Volume actually delivered on edge <code>e</code> under scenario <code>ω</code> ($\leq$ contract, $\leq$ surviving capacity).
<code>paid_volume[e,ω]</code>	Second-stage	Volume billed under take-or-pay terms ($\geq$ 70% of contract, $\geq$ delivered).
<code>flow[e,ω]</code>	Second-stage	Flow on non-shipping edges (pipelines, port-to-refinery, domestic).

Variable	Stage	Meaning
spr_draw[e,ω]	Second-stage	Strategic reserve drawdown on scenario ω (zero-constrained at baseline).
spot[r,ω]	Second-stage	Spot-market purchases delivered directly to refinery r.
shortfall[r,ω]	Second-stage	Unmet demand at refinery r under scenario ω.
VaR, z[ω]	Auxiliary	Rockafellar–Uryasev linearization variables for the CVaR term.

5.2 Constraints (as implemented)

- Contract capacity: contract[e] cannot exceed the edge's undisrupted capacity.
- Delivered ≤ contract, and delivered ≤ surviving (disruption-degraded) capacity, per scenario.
- Take-or-pay floor: paid_volume[e,ω] ≥ 0.7 × contract[e] and ≥ shipping_flow[e,ω] mirrors real term-crude contracts and makes over-committing to a chokepoint-fragile route a genuine, scenario-dependent cost.
- Non-shipping edge capacity (pipelines, port-to-refinery, domestic, SPR links), including a shared-capacity constraint where two edges draw on one physical pipeline.
- Baseline SPR draw fixed to zero the reserve is for disruptions, not routine daily supply.
- Supplier export cap (imported and domestic, the latter capped by real regional production, not just pipeline capacity).
- Port balance and port throughput inflow to a port (including domestic pipeline arrivals) must be able to leave it.
- Refinery demand balance: contracted delivery + SPR draw + spot purchase + shortfall = demand, per refinery per scenario.
- CVaR linearization: $z[\omega] \geq \text{scenario_total_cost}[\omega] - \text{VaR}$, $z[\omega] \geq 0$.

One constraint from the project's original one-page spec (SPEC.md) refinery grade compatibility, preventing a refinery from being assigned crude its Nelson complexity cannot process is not yet implemented, even though grade and grade_preference fields exist in the data. This is a genuine, previously undisclosed gap against the team's own spec, and is listed as the top near-term item in Section 10.

5.3 Objective

$$\min \sum_{\omega} \pi_{\omega} \cdot \text{scenario_total_cost}(\omega) + \lambda \cdot \text{CVaR}_{\alpha}(\text{total_cost})$$

where scenario_total_cost sums shipping/pipeline unit cost, spot premium, and shortfall penalty; $\text{CVaR}_{\alpha} = \text{VaR} + (1/(1-\alpha)) \cdot \sum_{\omega} \pi_{\omega} z[\omega]$, with $\alpha = 0.90$ by default. $\lambda = 0$ is the risk-neutral baseline; $\lambda > 0$ trades a small increase in expected cost for a reduction in worst-case (tail) cost, and is exposed to the user as a single slider in the Details & Charts tab. This mechanism was itself caught and fixed by the test suite: an early build gave every variable independent per-scenario freedom with no shared first-stage decision, so λ produced a byte-identical solution regardless of its value the take-or-pay constraint above is what makes risk-aversion actually change the answer, and a regression test now asserts this directly.

6. Disruption Scenario Modeling

scenario_engine.py degrades a shipping edge's capacity by severity × chokepoint_exposure, where exposure is a per-edge fraction (not a binary flag) reflecting how much of that specific route's volume actually transits the named corridor e.g. UAE's Fujairah terminal sits outside the Strait of Hormuz, so its exposure is set below 1.0 even though most of the country's crude originates inside the Gulf.

Scenario	Corridor	Severity	Planning Probability
Baseline		0%	55%
Hormuz 50%	Strait of Hormuz	50%	25%
Red Sea / Suez 80%	Red Sea / Suez	80%	15%
Hormuz 100%	Strait of Hormuz	100%	5%

These probabilities are documented, illustrative planning assumptions not a geopolitical forecast and are exposed as such in the app's own metadata.

7. Real-World Validation

Checked via live web search dated 21 July 2026 against reporting on the actual, ongoing Strait of Hormuz closure (full citations in KNOWN_LIMITATIONS.md §8a), not recalled from training data:

Metric	This Model	Real-World Reported
Hormuz-routed import share	48.2% (exposure-weighted)	40–52% (multiple sources)
Capacity lost under full closure	48.2% (2,380 of 4,936 kbpd)	“over 40%” of crude flows (OilPrice.com)
Strategic reserve cover	5.33 MMT, ≈9.5 days at full fill	5.33 MMT, “9–10 days of cover” (independent report)
Diversification direction	Risk-averse plan shifts contracts away from Hormuz-exposed suppliers	India secured ~70% of imports outside Hormuz by 11 Mar 2026 (Petroleum Ministry)

A second independent analog of the ongoing Red Sea/Houthi shipping crisis from November 2023 to the present was compared to the redsea_suspend scenario. It showed similar trends: actual Gulf to India freight rates increased by 35 to 40%, and India's import sources moved toward Brazil and the US during this crisis. This aligns with the diversification trend that the optimizer suggests for risk-averse conditions. The spikes in freight rates and insurance have no equivalent in this model (see Section 9).

What this model does not cover, simply put, includes the IEA's coordinated release of 400 million barrels, the impacts on GDP, the rupee, and capital flows, the losses from price-controlled retail fuel for Indian oil marketing companies, and the human toll from the ongoing conflict. The model's claim is narrower and more defensible. It accurately calculates the supply network and clearly states where it does not address other factors.

8. Engineering Rigor & Testing

The repository carries 111 automated tests across eight test files, covering data integrity (network.json well-formedness), scenario engine boundary conditions, optimizer constraint satisfaction on the solved model (not just solver-reports-optimal), signal extraction edge cases, briefing generation, maritime routing, the animated map, and full end-to-end UI paths via Streamlit's AppTest framework.

An independent adversarial audit of the solved model not just a code read found three high-severity issues, all since fixed and locked in with regression tests:

Issue	Root Cause	Fix
Domestic supply stranded at ports	~450 kbpd of indigenous crude reached a port with no onward edge counted by either the optimizer or scenario engine invisible to both.	Domestic pipeline inflows are now included in port balance/throughput on both sides.
SPR drained on an ordinary day	SPR was priced cheaper than routine spot crude with no depletion cost, so the model used it as a free daily supplier even at baseline.	Baseline SPR draw is now constrained to zero; reserves are reserved for disruptions.
Domestic production caps missing	Only pipeline capacity constrained domestic supply, not real regional field output, letting regions be drawn 2–3× their actual production.	A per-region production cap constraint was added (domestic_supplier_cap).

This is presented as a strength of the engineering process, not a caveat: the model was stress-tested against its own solved output, real defects were found, and every one of them is now a permanent, named regression test rather than a silent fix.

9. Known Limitations Disclosed, Not Discovered

Every item below is a deliberate scope cut or disclosed approximation, documented in full in KNOWN_LIMITATIONS.md before any reviewer could ask about it.

Limitation	Detail
Single-period model	One representative day per scenario no multi-day SPR depletion or replenishment schedule.
No rerouting-with-delay	alt_corridor / alt_transit_days are recorded on every shipping edge but not consumed by scenario_engine.py or optimizer.py; recovery happens only by reallocating volume to other edges, never by rerouting the same shipment onto a longer path.
No disruption-time freight/insurance spike	Freight cost is a static function of base transit days; real chokepoint crises raise freight and war-risk insurance 15–40% on affected routes even when they keep sailing.
Fairness objective cut from scope	CVaR redistributes shortfall pain across refineries but does not guarantee an equitable minimum service level; a fairness/max-min extension was cited in the original plan but not built.
Refinery grade-compatibility constraint not implemented	The team's own SPEC.md calls for it; grade and grade_preference fields exist in network.json but are not yet enforced as a solver constraint (Section 5.2).
SPR inventory sourced from PIB/Wikipedia, not PPAC	Strategic reserve figures sit under a separate MoPNG program outside PPAC's Ready Reckoner scope and could not be cross-checked the same way supplier shares were.
Fiscal-year vs. calendar-year mismatch	Country-wise import shares are exact DGCI&S calendar-2025 percentages applied to PPAC's fiscal-year 2025-26 volume total; the periods overlap nine months but are not identical (cross-checked within 0.3% on two independent metrics).

10. Future Roadmap, In Priority Order

#	Item	Why This Order
1	Multi-day SPR dynamics	Turns the single “day 1” snapshot into a real depletion-and-replenishment schedule directly closes a disclosed limitation with moderate effort.

#	Item	Why This Order
2	Rerouting-with-delay	Consumes alt_corridor / alt_transit_days data that already exists in network.json; lets the optimizer choose a slower, safer path instead of only reallocating volume.
3	Refinery grade-compatibility constraint	Closes the one gap against the project's own model spec (Section 5.2) cheap to implement, data already exists.
4	Live signal ingestion	Replaces curated headlines with real RSS/news-API monitoring, and layers in AIS vessel tracking where feasible.
5	Fairness objective	Adds the cited but unbuilt equitable-service-level term so shortfall is distributed fairly across refiners, not just minimized in aggregate.
6	True multi-agent reasoning	Lets an LLM reason over the knowledge graph and propose actions rather than templating the optimizer's output today's pipeline is agent-flavored, not yet agentic; the most ambitious item, listed last deliberately.

11. Deliverables & Repository Structure

PS2 Expected Deliverable	Status	Location
Working Prototype	Complete	app.py run with: streamlit run app.py
Architecture Diagram	Complete	Figure 1 (Section 3) and slide 4 of the presentation deck
Presentation Deck	Complete	PS2_Digital_Twin_Presentation.pptx
Demo Video	Script prepared	DEMO_VIDEO_SCRIPT.md (3–4 minute walkthrough)
Detailed Report	This document	PS2_Digital_Twin_Report.docx

Core repository contents: app.py (dashboard), optimizer.py (LP/CVaR), scenario_engine.py, signal_extractor.py, briefing_generator.py, build_network.py, animated_map.py, llm.py; data/network.json (the generated knowledge graph); tests/ (111 tests across 8 files); and the documentation set SPEC.md, KNOWN_LIMITATIONS.md, DATA_SOURCES.md, TESTING.md, DEVELOPMENT_LOG.md, AUDIT_FINDINGS.md which together record every data source, every modeling assumption, and every bug found and fixed over the course of the build.